



UNIVERSITY
INSTITUTE
OF LISBON

A Cross-Layer SDN Orchestration Architecture for Stateful Edge Services

Pedro Ojo Oriakhi

Master's in Telecommunications and Computer Engineering

Supervisors:

Ph.D. Professor José Moura, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Ph.D. Professor Pedro Ramos, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Month, Year

[This page is intentionally left blank.]



TECHNOLOGY
AND ARCHITECTURE

Department of Information Science and Technology

A Cross-Layer SDN Orchestration Architecture for Stateful Edge Services

Pedro Ojo Oriakhi

Master's in Telecommunications and Computer Engineering

Supervisors:

Ph.D. Professor José Moura, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Ph.D. Professor Pedro Ramos, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Month, Year

[This page is intentionally left blank.]

Write here your dedication.

[This page is intentionally left blank.]

Acknowledgments

Write here your acknowledgements and, if applicable, any grants or sources of funding.

[This page is intentionally left blank.]

Resumo

Palavras-Chave:

[This page is intentionally left blank.]

Abstract

Write here the abstract in English. There is a 250 word limit.

Keywords: *Keywords; separated; by; semicolons*

[This page is intentionally left blank.]

Contents

Acknowledgments	iii
Resumo	v
Abstract	vii
List of Figures	xi
List of Tables	xiii
List of Acronyms	xv
Chapter 1. Introduction	1
1.1. Context, Motivation, and Problem Statement	1
1.2. Objectives	4
1.3. Research Questions	4
1.4. Research Methodology	5
1.5. Main Contributions of this Dissertation	6
1.6. Dissertation Structure	7
Chapter 2. Background and Related Work	9
2.1. Review Methodology	9
2.2. Platform Technology Background	10
2.2.1. Edge Computing	11
2.2.2. SDN and OpenFlow	11
2.2.3. Containerisation and Docker	12
2.2.4. NoSQL and MongoDB	12
2.3. Monitoring & Telemetry	13
2.4. Auto Scaling	15
2.5. Load Balancing on SDN	17
2.5.1. Selection and Backend Membership	19
2.5.2. Admission as a Lifecycle Interface	19
2.6. SDN-Based Orchestration Platforms	20
2.7. Synthesis and Research Gaps	22
Chapter 3. Architecture and Design of the Proposed Solution for Edge Services	25
3.1. Design Requirements	25
3.2. Architecture	26
	ix

3.2.1.	Two-Domain Organisation	26
3.2.2.	Virtual IP Data Paths	26
3.2.3.	Control and Observation Paths	27
3.3.	Distributed Elastic Allocation of Resources	28
3.3.1.	Scaling Policy	28
3.3.2.	Services	30
3.3.3.	Data	30
3.4.	Monitoring and Decision Engine	31
3.5.	Control Workflow	32
Chapter 4.	Implementation of Proposed Solution	33
4.1.	Experimental infrastructure	33
4.2.	Distributed Elastic Allocation of Resources	33
4.2.1.	Services	33
4.2.2.	Data	33
4.3.	Monitoring and Decision Engine	33
4.4.	Control Workflow	33
4.5.	Implementation Validation	33
Chapter 5.	Experimental Results	35
5.1.	Experimental Setup	35
5.2.	Telemetry Delivery Semantics (RQ1)	35
5.3.	Bottleneck-Aware Scaling Action (RQ2)	35
5.4.	Readiness Admission to Traffic (RQ3)	35
5.5.	Network Performance	35
5.6.	Scalability Analysis	35
5.7.	Discussion: Synthesis Across Interfaces	35
Chapter 6.	Conclusions and Future Work	37
6.1.	Conclusions	37
6.2.	Research Contributions	37
6.3.	Limitations	37
6.4.	Future Research Directions	37
References		39

List of Figures

Figure 1.1	Global internet traffic between 2020 and 2025. Source: Statista, based on ITU data [1].	1
Figure 1.2	The demand-to-capacity chain: from a demand shift to usable capacity, with the three interfaces studied in this dissertation highlighted.	3
Figure 1.3	Design Science Research Methodology	5
Figure 2.1	Software-Defined Networking architecture: the control plane is decoupled from the data plane and programs the forwarding devices through the OpenFlow protocol.	12
Figure 2.2	MongoDB replica set: a primary member serving the clients and secondary members to which the data is replicated.	13
Figure 2.3	Load balancing in an SDN network: the controller installs forwarding rules that distribute client traffic among the available backends.	19
Figure 3.1	Architecture of the proposed platform: two network domains whose clients, data planes, edge servers, storage tiers, and aggregators are coordinated by peer SDN controllers, with host-local control paths and an emulated wide-area data link between the domains.	26
Figure 3.2	Double-VIP routing sequence: a client request addressed to <code>VIP_SERVER</code> triggers backend selection in the controller, and the edge server's storage access through <code>VIP_DATA</code> triggers the selection of a replica-set member.	27
Figure 3.3	Control and observation paths: edge servers and storage members publish observations to the per-domain telemetry aggregator, whose windowed summaries reach both peer controllers through a configurable delivery interface.	28

[This page is intentionally left blank.]

List of Tables

Table 2.1	Search-term families by domain.	10
Table 2.2	Selection criteria.	10
Table 2.3	Evidence for the link between monitoring and decision in the reviewed literature.	15
Table 2.4	Evidence on capacity-action selection in the reviewed literature.	18
Table 2.5	The three interfaces of the demand-to-capacity chain and the questions they leave open.	23

[This page is intentionally left blank.]

List of Acronyms

SDN: Software-Defined Networking

VIP: Virtual IP

OVS: Open vSwitch

ZMQ: ZeroMQ

DSRM: Design Science Research Methodology

[This page is intentionally left blank.]

CHAPTER 1

Introduction

Studies have shown that edge web services encompassing routing, monitoring and auto-scaling can suffer from coordination delays that arise from the independent control loops through which each module operates. This accumulation of handoff delays between components degrades orchestration efficiency and reliability which can potentially reduce service quality.

In this chapter the context, motivation and problem statement are explained in 1.1 with the objectives presented in 1.2. The research questions that emerged are available in 1.3. The methodology used to develop the thesis is detailed in 1.4, the main contributions in 1.5 and finally the outlining of the rest of the 6 chapters of the thesis are presented in 1.6.

1.1. Context, Motivation, and Problem Statement

In the year 2025 global internet traffic surpassed 7 exabytes (1 exabyte = 1 000 000 000 gigabytes), this corresponds to a more than double increase in the amount of internet traffic generated since 2020 across fixed and mobile networks, this is represented in Figure 1.1. [1].

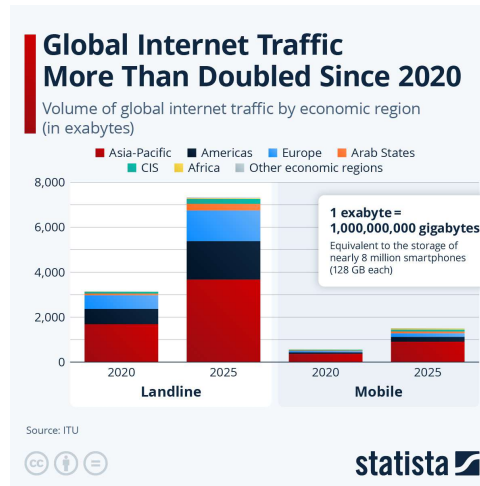


FIGURE 1.1. Global internet traffic between 2020 and 2025. Source: Statista, based on ITU data [1].

This growth has been driven by many technological advancements some of which, have in turn galvanized the emergence of real-time interactive and data-intensive applications among others. Examples include web streaming and cloud gaming, which are both services where users expect sub-second latency and high resolution multimedia, highlighting the

need for fast and capable data processing. These are just a few examples that demonstrate the demand of infrastructure that is able to deliver performance with high availability and reliability at global scale, in other words regardless of where the end users are located.

Cloud computing is and has been the [2] backbone that enables these applications to operate at a global scale. It offers automatic scaling, both horizontally and vertically, a pay-per-use model and among other aspects, [2] a globally distributed offering and availability that has established it as the foundational layer that supports modern applications and services.

Yet, in parallel, a growing share of internet traffic now originates from mobile devices, which have become the main gateway to internet access [3]. Much of this access is therefore generated and consumed at the network edge, far from centralised cloud data centres. As presented in [4], [5] edge computing brings the computational resources to the network edge therefore closer to the end-users, allowing for much lower latency in delivering services and applications to users by leveraging its geo-distributed nature. Beyond lower latency, edge computing's inherent geo-distributed servers dispersed geographically and close to users relieves bandwidth pressure on the core network, since data can be processed and stored locally, at the edge, for quicker responses, only a subset of it is sent to the cloud [4], freeing up backbone available bandwidth.

Despite these advantages, this geographical distribution also translates into less capacity per location and greater complexity in managing the distributed edge servers. As Satyanarayanan puts it, “the dispersion inherent in edge computing raises the complexity of management considerably” [5]. This management complexity stems from the dispersion of the edge servers across many sites, aggravated by considerable variation in user demand over time and by the heterogeneous resource requirements of the requests served. Luo et al. further highlight that, without efficient resource scheduling, this distributed nature leads to resource waste [6]. Resource management therefore becomes increasingly important, since the limited capacity of each location must be adjusted over time to suit the constant fluctuation of user demand.

The resource management problem gains more relevance when the services themselves are stateful. Unlike stateless microservices, which can be replicated freely and absorb traffic immediately, stateful services in a geo-distributed edge deployment depend on data held close to the compute: a content item ingested at one site is served from the database of that site. In distributed deployments, such data is routinely replicated across sites for locality and robustness, but replication is neither instantaneous nor free: a request reaching a server whose local copy is still synchronising must wait for it to catch up, and maintaining copies consumes storage and replication bandwidth [7], [8], [9].

Scale-out alone does not guarantee usable capacity: the orchestration system must detect when data must be brought closer, deliver that information to the decision point, provision the right kind of capacity, and redirect traffic to it, and each of these steps

introduces delay that worsens resource orchestration and can, in turn, lead to user service quality degradation.

This sequence forms what this dissertation calls the demand-to-capacity chain: the default path that an orchestration system must traverse to convert a change in overall user demand into capacity that serves that demand. A demand shift is first observed by monitoring and telemetry, which deliver the evidence to a decision point; the decision point selects a capacity action, such as scaling compute or scaling storage; the action provisions the new capacity; the new capacity attains a readiness state and is admitted to traffic, which is the point at which it becomes usable. The chain is therefore a timeline through the orchestration system, and certain steps are separated from the next by an interface at which delay or loss of evidence can accumulate.¹ Three of these interfaces, more specifically how demand evidence reaches the decision, which capacity action the decision selects, and when a ready backend is admitted to traffic are the subject of this thesis. Figure 1.2 illustrates the chain.

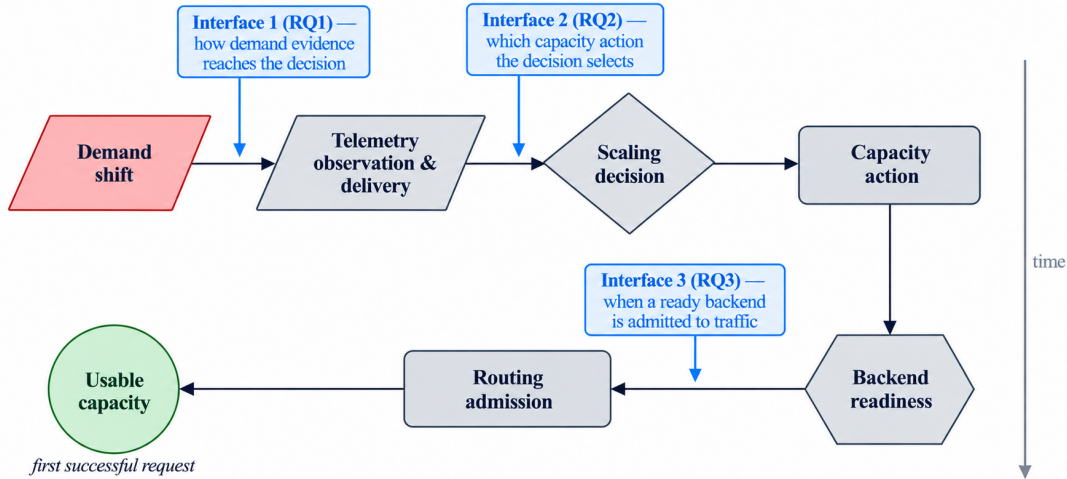


FIGURE 1.2. The demand-to-capacity chain: from a demand shift to usable capacity, with the three interfaces studied in this dissertation highlighted.

The steps described above must be coordinated under the time pressure of an ongoing demand shift. In practice, however, they are performed by separate components: a monitoring system collects metrics, an auto-scaler provisions capacity, and a load balancer redirects traffic, each running under its own control loop. The information needed to respond to demand must therefore cross from one component to the next, and the delays accumulated at these handoffs determine how quickly newly provisioned capacity can actually serve traffic. In the reviewed literature, these handoffs are acknowledged in passing

¹Throughout this dissertation, the *demand-to-capacity chain* denotes the full timeline from a demand shift to usable capacity; an *interface* is a boundary between two steps of the chain at which two functions meet; a *handoff delay* is the latency added at an interface when the functions on either side run under independent control loops; and the *coordination gap* is the accumulated effect of these handoff delays along the chain.

but never isolated or varied as an independent variable [10, 11]. This thesis investigates them directly through two peer Software-Defined Networking (SDN) controllers, one per network domain. Within each controller, telemetry consumption, scaling decisions, and traffic routing run in the same process. Telemetry production and aggregation remain external to the controller, making the delivery of aggregated demand evidence an explicit and configurable interface. Together with explicit lifecycle hooks for capacity action and traffic admission, this arrangement allows each selected interface to be measured under controlled conditions.

Along this chain, three interfaces are studied in a stateful service deployed across two geo-distributed sites: how demand evidence is delivered to the controller (telemetry delivery semantics), which capacity action the controller selects in response to the observed bottleneck (compute or storage scale-out), and when ready capacity is admitted to traffic (readiness admission).

The thesis does not claim that this SDN-based architecture is superior to existing orchestration platforms such as Kubernetes or NFV MANO, nor that the platform mechanisms generalise beyond the tested infrastructure; it claims only that these three interfaces are measurable and, within the reviewed literature, not previously characterised in isolation, and that varying each independently reveals which dimensions matter and under what conditions.

1.2. Objectives

The main objective of this thesis is to study the coordination gaps that arise between independent modules with their own control loops and to contribute to more efficient resource orchestration for stateful edge services, by characterising a specific and less examined part of this problem: the handoff delays that accumulate at the interfaces between telemetry delivery, capacity action selection, and traffic admission under varying demand shifts.

The investigation rests on the idea that running telemetry consumption, scaling decisions, and routing within each controller process keeps their controller-side coordination in-process while retaining telemetry delivery as an explicit, configurable boundary. Along with explicit lifecycle hooks for scaling and admission, this arrangement exposes the selected interfaces to direct measurement.

Because the components share one process and are therefore coupled, making each interface independently configurable is itself a deliberate design choice: the platform can change the delivery of demand evidence, the scaling action selected, or the admission of ready capacity while keeping the others fixed, and measure the effect of that single change.

1.3. Research Questions

This thesis seeks to answer the following three research questions:

- **RQ1** – How do verified event-preserving, delayed event-preserving, and latest-state telemetry delivery semantics affect overload observability, scaling response, and transient service quality in a stateful edge service?

- **RQ2** – Under compute-bound and data-access-bound demand, how does bottleneck-aware scale-out, compared with fixed single-tier policies, affect targeted-tier relief, service recovery, and resource efficiency?
- **RQ3** – For newly provisioned compute backends satisfying an identical servability criterion, how do lifecycle-driven and periodic readiness-admission mechanisms affect the time from verified readiness to routable and usable capacity, and what does the faster mechanism lose when its readiness notification is lost and spare capacity is scarce?

1.4. Research Methodology

The work presented in this dissertation follows the Design Science Research (DSR) methodology [12], [13] proposed in [14], and demonstrated in Figure 1.3.

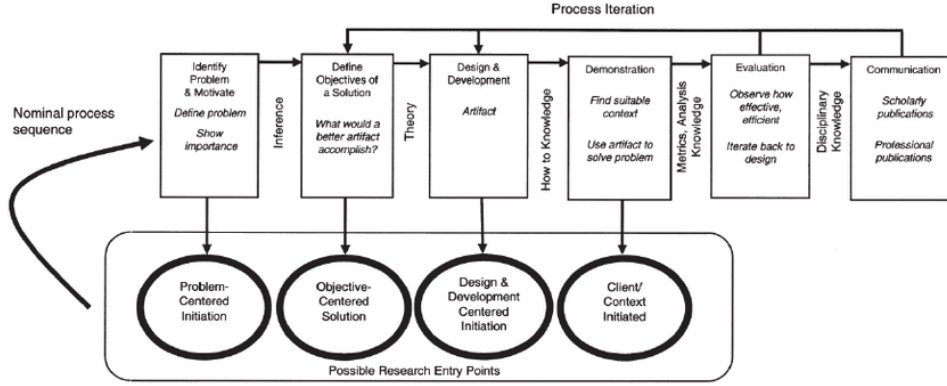


FIGURE 1.3. Design Science Research Methodology

Design science produces and evaluates purposeful artefacts that address certain real-world problems, the research contribution corresponds to the knowledge gained by designing the artefact and evaluating it against defined objectives. In Design Science Research Methodology (DSRM) the research is organized into six sections: problem identification and motivation, definition of the objectives of a solution, design and development, demonstration, evaluation, and communication.

This dissertation follows a problem-centred approach where the problem motivates the research and is defined as the coordination delays that accumulate between independently operating control loops in edge orchestration of services, which is justified by the state of the art reviewed in 2. The six activities unfolded as follows.

- (1) **Problem identification and motivation.** Edge services whose monitoring, routing and scaling modules operate as independent components accumulate handoff delays between control loops, degrading orchestration efficiency. The problem is introduced in 1.1, and the review of the state of the art in 2 establishes both that these functions are typically studied in isolation and that single-process architectures with independently configurable interfaces are less commonly present in the reviewed literature, which motivates and delimits the problem.

- (2) **Objectives of a solution.** As defined in 1.2, the solution is an experimental platform, not a novel algorithm, whose purpose is to make the interfaces of the demand-to-capacity chain measurable, so that their impact on resource orchestration can be evaluated through measurement and better understood.
- (3) **Design and development.** The artefact is the single-process platform described in 3 and 4: two peer SDN controllers (OS-Ken/OpenFlow), one per network domain, each hosting controller-side telemetry consumption, elasticity decision-making and the double-VIP routing model, with Docker containerised service backends and a MongoDB replica set per domain serving as the storage tier, each scaled in and out through MongoDB native replication. The technology choices are justified by their utility for the platform: SDN provides a single programmable control plane through which routing behaviour can be re-programmed directly in a central manner, Docker enables fast provisioning and teardown of service backends, and MongoDB for native replica-set replication and concurrent access.
- (4) **Demonstration.** The artefact was deployed as a running system in an emulated edge environment with two simulated sites, and was driven with scheduled workload phases reproducing demand shifts in order to demonstrate that the platform operates end-to-end from demand observation through scaling action to traffic admission, with the tested interface varied across runs of each experimental campaign for evaluation.
- (5) **Evaluation.** Each research question was evaluated in a dedicated experimental campaign reported in 5, with controlled reference conditions holding the other two interfaces constant. The evaluation follows a pre-registered measurement and statistical procedure: each experiment is composed of multiple independent runs, each treated as an experimental unit, driven by a scheduled open-loop workload driver that preserves the offered load; the primary comparisons are detailed in 5.1. Internal, external and repeatability validity threats are discussed in 6.3.
- (6) **Communication.** The communication of the problem, the artefact and the findings is accomplished through this dissertation itself and its public defence, as well as through any resulting publications.

1.5. Main Contributions of this Dissertation

This dissertation makes the following contributions:

- (1) An experimental characterisation of telemetry delivery semantics (event-preserving, delayed event-preserving, and latest-state) and of their effects on overload observability, scaling response, and transient service quality (RQ1).
- (2) An experimental characterisation of bottleneck-aware capacity action selection: how deciding between compute and storage scale-out from the observed bottleneck, compared with fixed single-tier policies, affects targeted-tier relief, service

recovery, and resource use under compute-bound and data-access-bound demand (RQ2).

- (3) An experimental characterisation of readiness admission that quantifies, under direct lifecycle notification versus periodic discovery, the time from verified readiness to routable and usable capacity, and the reliability cost the faster mechanism incurs when its readiness notification is lost and spare capacity is scarce (RQ3).
- (4) A comparative synthesis of the observation, action-selection, and readiness-admission interfaces under their respective controlled workload regimes, identifying which effects were robust, workload-bound, or inconclusive.

These contributions are claimed within the evaluated two-domain testbed and workload families. The evaluation does not allow us to claim that the single-process arrangement is superior to other orchestration platforms, nor was it conducted with such intent.

1.6. Dissertation Structure

This dissertation is organised into six chapters:

- **Chapter 1** introduces the research, presenting its context, motivation and problem statement, the objectives, the research questions that emerged, the methodology that guided the work, and the main contributions.
- **Chapter 2** reviews the background and related work. After presenting the methodology used to conduct the review, it surveys the platform technologies on which the proposed solution rests (edge platforms, SDN and OpenFlow, containers, and NoSQL document stores), then monitoring and telemetry, auto-scaling, load balancing on SDN, and resource orchestration on SDN, examines the default separation of monitoring, routing and scaling into independent components observed across these areas, and concludes with the research gaps that motivate this dissertation.
- **Chapter 3** presents the architecture and design of the proposed solution for edge services: the design requirements, the SDN-based architecture, the distributed elastic allocation of compute and data resources, the monitoring and decision engine, and the control workflow.
- **Chapter 4** describes the implementation of the proposed solution: the experimental infrastructure, the implementation of the elastic allocation of compute and data resources, the monitoring and decision engine, the control workflow, and the validation of the implementation.
- **Chapter 5** presents the experimental results. Following the demand-to-capacity chain, the experimental campaigns address telemetry delivery semantics (RQ1), bottleneck-aware scaling action selection (RQ2), and readiness admission to traffic (RQ3), followed by resource orchestration and scalability analysis, and a synthesis across the three interfaces.

- **Chapter 6** concludes the dissertation, restating the findings and the contributions, discussing the limitations of the work, and outlining directions for future research.

CHAPTER 2

Background and Related Work

This chapter presents the background and related work that motivated the research, selected through the search strategy described in 2.1. The review is separated into two parts. First, the technologies and concepts used to build the experimental platform are explained and surveyed, each introduced for what it is and why it suits the edge scenario. These are, more specifically, edge computing, SDN and OpenFlow, containerisation, and NoSQL document stores.

The layer that binds these technologies and concepts into a running service, known as resource orchestration, is also reviewed and alongside the three functions it covers, which are monitoring and telemetry, auto-scaling, and load balancing. Then the focus is drawn onto the interfaces between them that are part of the demand-to-capacity chain to evaluate how demand shifts are detected, which capacity action is selected when several are available, and how readiness admission is handled.

Within each area, the review examines whether and how the interfaces between these functions are considered, studied, evaluated, or mentioned; whether the reviewed work provides evidence that the delays and losses at these handoffs cause orchestration or service-quality problems; and what ideas each field offers about them, with the aim of establishing a solid ground for the evaluation of the three interfaces of the demand-to-capacity chain.

This examination grounds the claim, introduced in 1.1, that these interfaces are acknowledged in the literature but have not been systematically isolated and studied. The orchestration platforms that provide the context for these functions are reviewed in 2.6, and the findings across all areas, together with the research gaps they define, are synthesised in 2.7.

2.1. Review Methodology

Searches were executed in Google Scholar and IEEE Xplore targeting recent literature, publications from 2016 onward with consideration for the number of citations. Works reached through Google Scholar were retrieved from IEEE Xplore, ACM Digital Library, Springer, and ScienceDirect. Each query resulted in roughly 30 papers evaluated for selection. Table 2.1 groups the search-term families by domain, and Table 2.2 states the selection criteria applied to the retrieved hits.

Screening and citation chaining produced the reviewed literature, organised by the interface each work informs: telemetry delivery, capacity action selection, readiness admission, and platform context. Sources used solely to explain the platform technologies

TABLE 2.1. Search-term families by domain.

Domain	Example search terms
Monitoring and telemetry	“SDN monitoring”, “SDN telemetry”, “push vs pull telemetry”, “monitoring delay SDN”, “container monitoring edge”, “edge platform monitoring”
Auto-scaling and data	“auto scaling SDN”, “scaling trigger”, “Kubernetes HPA end-to-end latency”, “application metrics”, “edge storage survey”, “edge data placement”
Load balancing	“load balancing SDN”, “handoff delay”
Platform context	“cross-layer orchestration”, “SDN resource orchestration edge”, “edge platform survey”, “geo-distributed edge”, “stateful edge web services”

TABLE 2.2. Selection criteria.

Inclusion	Exclusion
Addresses at least one of the three interfaces in software-defined, containerised, or edge settings	Focus outside the orchestration of service capacity (e.g., physical layer, security-specific, domain applications without an orchestration contribution)
Surveys and taxonomies that frame one of the domains	Duplicates
Seminal works reached by citation chaining	Non-English or full text unavailable
Full text freely available or accessible under ISCATE’s scientific access licences	
At least ten citations, or published within the last five years and retained on topical relevance	

in 2.2 are treated as background rather than as independent evidence of an RQ gap. The synthesis is recorded in 2.7.

2.2. Platform Technology Background

This section presents and explains the four building blocks that define the testbed evaluated in this thesis: edge computing, SDN with OpenFlow, containerisation, and MongoDB.

Each subsection states what the building block is and why it matters for the thesis scenario. How the building blocks are applied is the subject of Chapters 3 and 4.

2.2.1. Edge Computing

Edge computing is the placement of computational and storage resources between end devices and centralised cloud data centres, close to where data is produced and consumed. It complements rather than replaces the cloud: latency-sensitive processing is shifted to the edge, while large-scale analytics and long-term storage remain in the cloud [4].

In essence, edge computing exists because centralised clouds alone cannot serve every workload well, and bringing computation closer to the user lowers the latency of interactive and data-intensive services, and processing data locally relieves the bandwidth pressure on the core network [5].

Despite the advantages of edge computing, as mentioned previously in 1.1, the inherent geo-distribution of resources increases their management and orchestration complexity. Resource heterogeneity, scarcity, and demand varying over time are all aspects that can degrade resource orchestration efficiency, making it a concept of continuous study and research [6, 15]. Scheduling that does not follow the workload leads either to over-provisioning, where resources are held but not used, or to under-provisioning, where service quality degrades [6]. In addition, Luo et al. identify the joint scheduling of communication, computing, and storage resources as an open direction for the field [6].

Managing these platforms remains an open challenge, and their combination with SDN is recorded as one of the continuing challenges [16]. The service evaluated in this dissertation runs in this environment: a stateful edge service whose capacity must follow fluctuating demand, which is the management problem the remainder of this chapter reviews.

2.2.2. SDN and OpenFlow

Software-Defined Networking separates the control plane, which decides how traffic is handled, from the data plane, which forwards packets, so that forwarding behaviour can be reprogrammed centrally from a controller instead of being configured device by device.

Baktir et al. survey how edge computing benefits from this decoupling and organise the gains into benefit areas such as centralised visibility and programmatic traffic management, arguing that SDN can act as the enabler that lowers the complexity barriers and enhances the flexibility of edge deployments [17].

The OpenFlow protocol implements the decoupling through match-action flow rules installed on the switches, and is, as reviewed by Waseem et al., the most extensively used interface for programming such networks [18].

In its OpenFlow form, a single controller re-programs the forwarding behaviour of the network by installing and updating match-action flow rules, so that network behaviour can follow the life cycle of the services it carries without reconfiguring the forwarding devices individually. Programmable forwarding therefore turns the admission of a newly ready backend into a controller decision rather than a fixed property of the network.

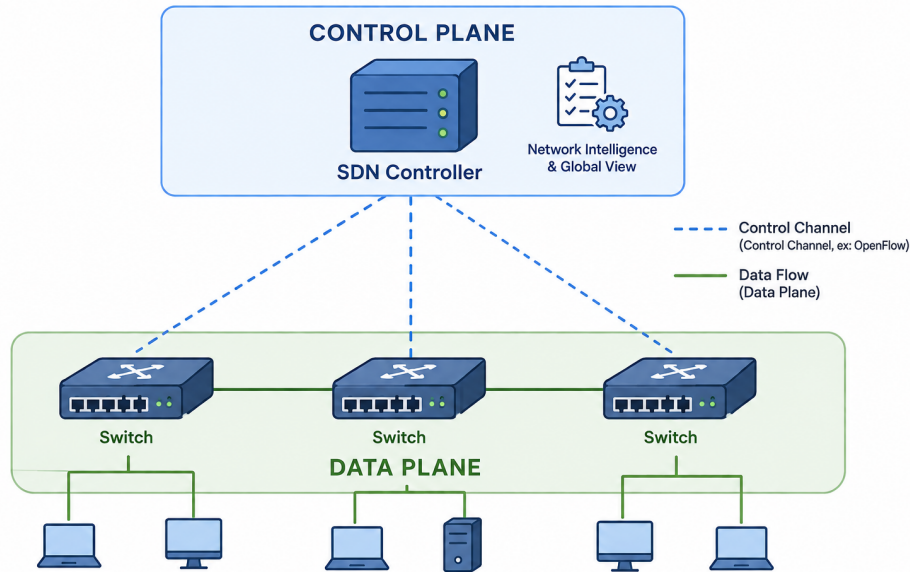


FIGURE 2.1. Software-Defined Networking architecture: the control plane is decoupled from the data plane and programs the forwarding devices through the OpenFlow protocol.

2.2.3. Containerisation and Docker

Containers package an application with its dependencies and share the host kernel, which makes them lightweight and quick to start up compared to virtual machines, properties that suit resource-constrained edge platforms. Docker is adopted as the platform's containerisation tool because it is the most widely used tool for building and running containers.

2.2.4. NoSQL and MongoDB

Document-oriented NoSQL stores organise data as schema-flexible documents rather than as rows of relational tables. Because related data is embedded in a single document and the model avoids joins and fixed schemas, records can be partitioned and replicated across servers, which is how these stores scale horizontally, whereas relational systems are traditionally scaled vertically by enlarging a single server. This model suits the heterogeneous data of edge and IoT services, whose structure varies between sources and evolves over time.

MongoDB is a document store of this family: it holds data as BSON documents within collections and provides a flexible schema. In MongoDB, redundancy is organised natively around the replica set: a group of `mongod` processes that maintain the same data set. One member is elected primary and serves the clients' writes, which it records in an operation log; the remaining members, the secondaries, replicate by applying that log asynchronously and autonomously. Members exchange heartbeats, and when the primary fails or becomes unreachable, the secondaries hold an election and promote one of their own, so the set keeps serving without operator intervention. Members can also be added

to and removed from a deployment while it keeps serving, which turns the replica set into a controllable unit of storage capacity rather than a fixed configuration.

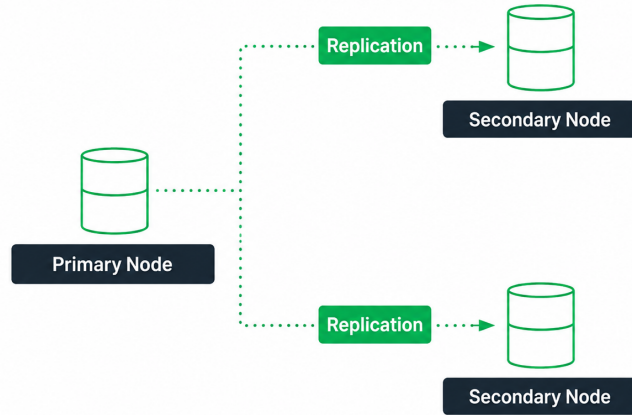


FIGURE 2.2. MongoDB replica set: a primary member serving the clients and secondary members to which the data is replicated.

This dissertation builds on that capability: the storage tier is scaled by adding and removing replica-set members while the set keeps serving, leveraging MongoDB’s native replication and election mechanisms.

2.3. Monitoring & Telemetry

Monitoring and telemetry observe the state of a running service and deliver that evidence to whoever acts on it. Monitoring is the collection of resource and load metrics, telemetry the transport that carries the collected data to the consumer, and together they form the first link of the demand-to-capacity chain: every later decision, on scaling or on routing, can act only on the evidence this link delivers, at the time it delivers it, and with whatever intermediate observations the delivery process preserves. In edge deployments the link matters doubly, because the evidence must travel from many dispersed nodes to the decision point, so any delay or gap in its delivery becomes delay or blindness in the orchestration response.

The state of the art in this area is built around a pipeline in which per-node collectors sample resource and load metrics, an aggregation layer summarises the samples, and the resulting series feed the consumers that make decisions. In orchestrators this pipeline is standardised around periodic scraping into a time-series store, as in OSM-based NFV for transparent VNF clusters, where the monitoring module polls the infrastructure into a Prometheus database [11].

Delivery is conventionally pull-based, the consumer requesting the latest sample at a fixed interval [19], and this choice alone determines much of what the consumer can know: between two collection instants the monitored state is unobserved, and what the pipeline

forwards between instants determines whether the consumer receives the complete history, a complete but delayed history, or only the latest state. These different forms are defined as event-preserving delivery, delayed event-preserving delivery, and latest-state delivery. Their effect on resource orchestration is part of the subject of study in this thesis.

The reviewed evidence spans three contexts in which this link has been studied: cluster-federation monitoring, network telemetry, and edge-platform observability. In each, the review asks whether the delivery mechanism itself is varied, what downstream consequence the variation is shown to produce, and what the work leaves as a given.

Within cluster-federation monitoring, the collection interval has been shown to be a genuine variable rather than a fixed parameter. AdapPF [20] varies the Prometheus scrape interval of a geo-distributed Kubernetes federation and measures the effect on a downstream consumer: under high load, a coarse sixty-second interval substantially degrades scheduling accuracy relative to a five-second interval, and an adaptive cadence restores it while reducing cross-cluster monitoring traffic. AdapPF is the only reviewed work that varies monitoring freshness and measures a downstream consequence, but it stops at a single consumer, a scheduler, and does not run over a stateful service.

The same line of work treats monitoring overhead as a recognised bottleneck in geo-distributed deployments, proposing aggregation so that monitoring can scale with the federation [21], aggregation being itself a form of summarisation that discards intermediate observations. Within the same platform context, Zhou and Yong study what to monitor rather than how it is delivered, comparing custom-metric autoscaling driven by application-level failure rates against conventional CPU-based autoscaling in Kubernetes, and find that metric choice changes scaling quality under load, albeit on a small single-service testbed [22].

A second context, network telemetry, converges on the same delivery mechanics from the network side. Yaseen reviews programmable network-wide monitoring and records that pull-based polling produces “visibility gaps” between collection instants, calling for monitoring to be “tightly integrated with network control and automation” [19]. Hung et al. present an SDN controller for 5G backhaul networks that closes a telemetry-and-routing loop, with telemetry collection delegated to decentralised microservices reached through an API, showing that monitoring and control functions can be combined at one control point, although the loop omits scaling altogether [23].

At the platform level, adjacent observability work exhibits the same open dimension: Usman et al., surveying observability of distributed edge and container-based microservices, leave the definition of appropriate measurement intervals as an open research topic [24]. None of these works runs over a stateful service, so they are read here for the delivery mechanisms they expose, not as direct precedents for the platform under study.

Table 2.3 summarises the strength of the evidence each of these works provides for the link under study.

TABLE 2.3. Evidence for the link between monitoring and decision in the reviewed literature.

Work	Strength	What it shows
Llorens-Carrodegua et al. (2021)	Measured	Orchestration monitoring polls into Prometheus; delivery semantics not varied
AdapPF (Huang & Pierre, 2023)	Measured	Freshness changes a scheduler’s accuracy; no stateful-service or admission trace
Huang & Pierre (2024)	Measured	Aggregation contains monitoring overhead but discards intermediate observations
Zhou & Yong (2024)	Measured	Metric choice changes scaling quality; the delivery mechanism stays fixed
Yaseen (2025)	Documented	Pull-based polling leaves visibility gaps; calls for monitoring integrated with control
Hung et al. (2022)	Measured	Telemetry-and-routing loop at one control point; scaling omitted
Usman et al. (2022)	Surveyed	Appropriate measurement intervals remain an open research topic
Belgaum et al. (2020)	Not addressed	A seventy-six-paper review of the adjacent load-balancing field lists no freshness issue among its open problems

Across the reviewed literature, therefore, the building blocks of the delivery question appear separately: AdapPF varies the interval that produces delay, Yaseen documents the gaps that polling leaves, and aggregation summarises away intermediate observations, yet no reviewed work distinguishes delayed event-preserving delivery from latest-state delivery, and none traces either regime through a stateful service’s scaling and traffic-admission path. This distinction is the gap this dissertation isolates and measures.

2.4. Auto Scaling

Auto-scaling adjusts the amount of allocated resources to the load that a service receives: when demand grows, additional capacity is provisioned, and when demand recedes, surplus capacity is released, so that the service keeps serving requests without holding resources it does not use. Auto-scaling strategies are commonly grouped into horizontal scaling, which changes the number of units, and vertical scaling, which changes the capacity of a single unit [25]. Horizontal scaling suits the edge scenario, and is the direction this dissertation follows.

In edge computing, a site’s capacity is bounded by the nodes deployed there, so enlarging an individual unit has a hard ceiling, whereas adjusting the number of units lets capacity grow and shrink as demand changes. When and how many resources to add or remove therefore determines both the quality of service during a demand shift and the efficiency with which scarce resources are managed.

Qu et al. classify the auto-scalers along the scaling indicator, the resource estimation method, and the timing of the action, and Taherizadeh and Stankovski organise the same concepts across edge application types and operational challenges. In both, the unit surveyed is the scaling of service instances, whether virtual machines or containers. In short, the main questions the field asks are when to scale and how many instances to add, and which kind of capacity is provisioned, when applicable is not among the classification dimensions [10, 26].

When SDN enters the loop, the traffic side is added, but the scaled resource remains the one the service function is built from: Llorens-Carrodegua et al. couple SDN-based traffic steering with horizontal scaling of virtualised network functions, so that a single controller re-programs the forwarding paths while instances of the same pre-dimensioned function pool are activated or deactivated. The decision reduces to whether and how many instances of that one function to activate, and the data tier those functions depend on is held fixed by the design. In a stateful edge service, however, that fixed tier is itself a live capacity: when the pressure sits in the data rather than in the function instances, adding instances cannot relieve it [11].

Predictive and learning-based approaches sharpen the same decision without widening it: Toka et al. formalise the Kubernetes horizontal autoscaler’s control model to anticipate demand for edge clusters [27], Ju et al. and Cheng et al. forecast workload to adjust replica counts ahead of demand, the latter jointly with placement [28, 29], and Li et al. couple scaling and placement in a single optimisation [30]. Across these systems the quantity that is forecast and adjusted is the number of compute replicas. The improvement they pursue is a better forecast of how many instances to run; which tier holds the limiting resource does not enter their problem.

Scaling one resource does not always make the service better, because the pressure may sit elsewhere. Nikraves et al. quantify what that default costs in a three-tier web service: the scaling decisions taken for the business tier turn out to be insensitive to database capacity, yet reducing database capacity raises the rate of SLA violations, so service quality can be governed by the tier that the scaling decisions ignore [31]. Dogani et al. record the same boundary across the systems they survey: adding resources does not necessarily raise throughput when the bottleneck lies elsewhere [32]. Among those systems, awareness of the effect is scarce and cross-service only: Waterfall, a machine-learning auto-scaler, accounts for how scaling one microservice can shift the bottleneck towards downstream services, and ATOM uses layered queuing networks to reduce bottleneck

shifts in model-agnostic controllers; both provisions remain compute-side, and no surveyed system selects a storage action.

The most recent systems sharpen the same boundary: PBScaler identifies the microservice whose degradation propagates through the call graph, ranks candidate bottlenecks, and optimises replica management around the identified bottleneck [33]. Bottleneck awareness therefore exists in the current state of the art, but the bottleneck it locates is always a microservice and the action it takes is always a compute-replica count, so the selection between compute and storage capacity remains outside the action space.

The closest reviewed precedent to selecting between resource tiers is the provisioning model of Li et al., which adds both cloud compute instances and data replicas in cloud-assisted edge systems, from a forecast of future workload built over historical workload data [34]. Both actions are available, but the trigger is a forecast rather than a live observed bottleneck, and the outcome is measured in tenancy cost and utilisation rather than in user service quality and resource management efficiency.

Pelle et al. keep function and state placement in sync through an integrated optimisation over an edge platform [9]. The mechanism is placement rather than live capacity action: state is moved to follow the functions, while the response to an observed bottleneck remains outside the optimisation.

The most recent survey converges on the same boundary: El Mrabet et al. extend the classification across cloud, fog, and edge, categorising the surveyed methods by their decision mechanism, scaling strategy, and deployment layer, and the provisioned tier is not among the classification dimensions [25].

Qu et al. describe the web application as split into application logic and a database tier, and record that “the database tier is often considered dynamically unscalable and ignored by auto-scalers” [10].

Overall, the literature demonstrates that the kind of capacity added under certain scenarios can have an impact, compute replicas cannot relieve pressure whose limiting resource lies in the data tier, and database capacity can govern SLA violations while a compute-tier scaling decision remains unchanged. Capacity actions are therefore not interchangeable merely because each adds resources.

Therefore in the reviewed studies, the choice between these actions remains an architectural given, settled by forecasts or by joint placement rather than selected at runtime from a live observed bottleneck. This dissertation isolates that selection, with storage scale-out among the actions the controller may select, and measures how the benefit divides across targeted-tier relief, service recovery, and resource use. How that decision reaches execution is examined, with the other interfaces, in 2.7.

2.5. Load Balancing on SDN

Load balancing distributes incoming traffic among a set of available backends so that no single backend is overloaded while others remain idle. In SDN networks this distribution is programmed centrally, where the controller installs forwarding rules that steer each

TABLE 2.4. Evidence on capacity-action selection in the reviewed literature.

Work	Strength	What it shows
Qu et al. (2018)	Surveyed	Canonical taxonomy of when and how many to scale; database tier ignored by the surveyed auto-scalers
Taherizadeh & Stankovski (2017)	Surveyed	Edge-specific taxonomy; scaling surveyed over service instances
Llorens-Carrodegua et al. (2021)	Measured	SDN steering coupled to horizontal VNF scaling; capacity-action type fixed
Toka et al. (2021)	Measured	Learning improves Kubernetes edge scaling; compute-only with the metrics pipeline taken as given
Cheng et al. (2023)	Measured	Instance counts and placement jointly optimised; provisioned unit remains compute
Nikraves et al. (2017)	Measured	Database capacity ignored by the business-tier autoscaler, yet it drives the SLA violations
Dogani et al. (2023)	Surveyed	Bottleneck-shift awareness exists, compute-only and cross-service; no storage action
PBScaler (2024)	Measured	Bottleneck-aware replica management; the bottleneck is a microservice, the action remains compute
Li et al. (2020)	Measured	Compute instances and data replicas provisioned together, but from a forecast, not a live bottleneck
El Mrabet et al. (2025)	Surveyed	Recent taxonomy; which resource is added is not among the classification dimensions

flow to a selected backend, and a selection policy decides which backend receives the next flow. The function matters at the edge because a newly provisioned backend only becomes usable capacity once traffic actually reaches it, so how backends enter and leave the pool of candidates is part of the problem itself.

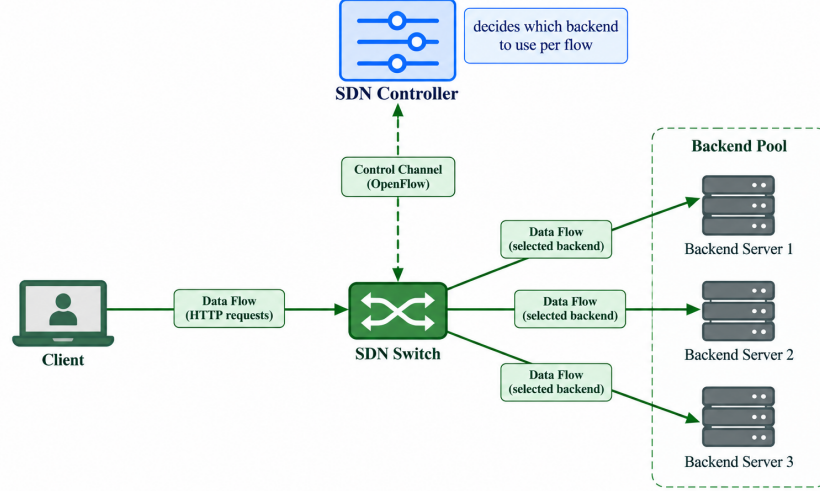


FIGURE 2.3. Load balancing in an SDN network: the controller installs forwarding rules that distribute client traffic among the available backends.

2.5.1. Selection and Backend Membership

At a given routing decision, a load-balancing policy must operate over the set of backends currently eligible to receive traffic. Treating this set as fixed for that decision is therefore a valid modelling boundary rather than a limitation of the selection policy. The SDN load-balancing literature accordingly concentrates on how to select a backend from the available candidates, using criteria such as load, path properties, and resource utilisation [35, 36].

The set of eligible backends may nevertheless change between routing decisions. This change does not alter the selection problem itself; it creates a separate lifecycle responsibility. A backend must be provisioned, become able to serve the application, have that state propagated to the routing plane, and be included in the eligible set before the next routing decision can select it. The interval between verified readiness and routing eligibility is therefore an admission delay, not a property of the backend-selection algorithm.

2.5.2. Admission as a Lifecycle Interface

Kurma, a load balancer for geo-distributed storage systems, is explicitly designed to operate at a granularity of seconds so that it can “work in tandem with modern elastic controllers, thereby reducing over-provisioning and SLO violations incurred during provisioning delays” [37]. The delay between provisioning and usable capacity is therefore a recognised cost, yet the interval in which readiness state propagates to the routing plane is not separated from the surrounding lifecycle.

The closest architectural antecedent is Llorens-Carrodegua et al., whose SDN redirector combines monitoring, auto-scaling, and load balancing for a transparent VNF cluster. Their system changes the number of active cluster members and removes flow rules for unhealthy instances using status obtained through the MANO monitoring path [11]. It

therefore demonstrates that dynamic membership is already a legitimate concern. Its evaluation, however, reports activation at the granularity of the integrated system, so the interval from verified application readiness to routing eligibility is not separated from the surrounding lifecycle.

Across this literature, therefore, the selection of a backend among eligible candidates is well characterised, and integrated systems implement broader membership transitions. What remains unisolated in the reviewed studies is the interval between a backend becoming ready and traffic actually reaching it. This dissertation isolates that interval: it varies readiness-admission coordination while holding the application-readiness criterion and selection function constant, and measures the time until newly provisioned capacity becomes selectable and serves traffic. Direct lifecycle notification and periodic discovery are the two operational realisations of that coordination interface.

2.6. SDN-Based Orchestration Platforms

Resource orchestration is the management layer that keeps a running service within its requirements by coordinating the compute, network, and storage resources of the infrastructure on which it runs. It is the activity that the technologies and tools reviewed in 2.2 are assembled to serve: edge sites supply the resources, SDN supplies the programmable connectivity, containers supply the unit of compute, and the document store holds the service state. That decision-making, and its execution, is orchestration, and at the edge it is the more demanding because each site holds scarce capacity and the demand on it fluctuates over time. Because the service is stateful, the capacity action is itself two-sided: compute replicas serve requests, while the data they serve lives in the storage tier, so scaling can address either side.

This dissertation studies the SDN-based form of this management layer, in which the coordination spans the SDN controller, the container runtime, and the data store, and the challenge is to keep these layers working together as demand moves across sites. The platforms reviewed here bind these layers with different degrees of coupling, and this section examines how closely the three functions of the demand-to-capacity chain, monitoring, scaling, and routing, are held together in each, using the terms interface and coordination gap as defined in 1.1.

Along the layers axis, edge orchestration is cross-layer by construction. Baktir et al. organise the gains SDN brings to edge computing into benefit areas such as centralised visibility and programmatic traffic management [17]; Wang et al. survey architectures in which SDN and NFV jointly coordinate placement, traffic, and caching across the network and service layers, showing that SDN’s role at the edge extends beyond forwarding into orchestration [38]; and Sofia et al. argue for context-aware cross-layer orchestration of containerised edge-cloud applications serving next-generation IoT services, treating contextual information such as location and data freshness as part of the orchestration input [39]. These works document architectures rather than the coordination inside a running system, so the interfaces between the functions fall outside their scope.

The surveyed architectures take concrete form in deployments that run SDN and container orchestration side by side as separate systems. Okwuibe et al. combine an OpenDaylight controller with Kubernetes for a containerised industrial-IoT video service, the component set closest to this dissertation’s. The two platforms are coupled through scripted triggers rather than shared state: a script watches the mobile core’s log for client handovers and triggers a Kubernetes migration of the service between MEC servers, and the service becomes reachable again only after a DNS update. The study reports the total migration latency and names the factors it bundles, from scheduler performance and the network fabric to container and application start-up, without attributing time to them individually, and it lists reducing these factors as future work. Its databases serve the power-measurement instrumentation rather than the service path, and the demonstration workload is stateless [40]. Peng et al. exercise SDN-based resource management in vehicular multi-access edge computing as task migration and radio slicing, treating telemetry as available input rather than as an interface to be studied, and without container orchestration or tier scaling [41].

A recurring minority of these platforms brings the functions together at the SDN control point itself. Llorens-Carrodegua et al. combine monitoring, auto-scaling, and load balancing in a single SDN redirector for a transparent VNF cluster, although metric collection and the lifecycle operations that feed and execute the decision remain external [11]; Hung et al. present an SDN controller for 5G backhaul that closes a telemetry-and-routing loop, with telemetry collection delegated to decentralised microservices reached through an API, omitting scaling altogether [23]. The first arrangement binds all three functions at one control point and the second binds two of the three, so combining the functions at an SDN control point is a feasible, precedented arrangement rather than an invention of this dissertation. Neither system, however, measures the delay that its coordination introduces.

At the geo-distributed scale, three systems coordinate the same functions by construction, and they delimit the claim of this dissertation. OneEdge keeps authoritative state at each edge site, reconciles it with a central aggregate through two-phase commits, and uses ZeroMQ (ZMQ) for communication among its distributed components, reporting the whole control plane’s deployment latency rather than the contribution of any single hand-off [42]. McK8s deploys four controllers on a management cluster as Kubernetes operators over custom resources, connects member clusters through a Cilium mesh, and measures inter-cluster latencies with Serf, treating those latencies as input to placement rather than as an effect to study [43]. Kurma instances exchange load, latency, and SLO-violation measurements among datacenters and redirect requests at second granularity while elastic controllers provision capacity, naming the stages of the provisioning delay, detection, acquisition, and warm-up, before treating them as one combined quantity [37]. The coordination works by construction in each, but the delay it carries is not isolated: the state

updates, exchanges, and redirects along those communication paths are not attributed to the individual handoffs between observing demand, deciding, and admitting capacity.

The related background is therefore organised in two modes. In the dominant mode, monitoring, scaling, and routing are implemented as separate components, each running its own control loop, so the evidence and commands that tie them together must cross component boundaries. The separation is visible even in the tools themselves: Cilic et al. evaluate Kubernetes, the lightweight distributions K3s and KubeEdge, and the ioFog platform in real edge execution environments, and find that the scheduling logic of these tools can be customised to a degree, but that scheduling from client QoS parameters still requires integration around them, since QoS tracking and re-scheduling are not built in; they conclude that further integration work is needed to adapt these cloud-designed tools to dynamic edge environments [44]. In a second, recurring mode, the same functions are kept together at an SDN-based control point or coordinated through shared state. What neither mode provides, however, is a controlled decomposition in which one handoff is varied while the others are held fixed. Prior systems measure complete control processes or coordinate them by construction; they do not isolate the contribution of telemetry delivery, capacity-action selection, or readiness admission to the demand-to-capacity path. This interface-level contribution is precisely the quantity this dissertation highlights.

SDN-based resource orchestration is therefore an established and feasible arrangement, and the coordination it rests on is a recorded open issue: Nain et al. list the coordinated management of resources across the SDN-based edge among the continuing challenges of the field [16]. The handoff delay studied in this dissertation is one measurable dimension of that coordination, and because the functions meet at boundaries that an SDN-based control point can expose as explicit, configurable interfaces, such a platform is a suitable experimental basis for evaluating the handoff delays between them. The platform built in this dissertation follows that arrangement and exposes telemetry delivery, capacity-action selection, and readiness admission as independently configurable interfaces, as presented in 3.

2.7. Synthesis and Research Gaps

The review travelled through the functions of resource orchestration, and each area concluded with the gap it leaves; read together, the four areas show the handoff delay at three levels of treatment. The delay is first acknowledged as a real cost: Kurma’s seconds-granularity operation aims to reduce the over-provisioning and SLO violations incurred during provisioning delays [37], Yaseen records the visibility gaps that pull-based polling leaves and calls for monitoring tightly integrated with control [19], and Cilic et al. conclude that integration work is still required around the scheduling logic of cloud-designed edge tools [44]. Second, where the delay is evaluated, it is evaluated only in composite: Llorens-Carrodeguas et al. report activation at the granularity of the integrated system, Okwuibe et al. report a total migration latency, OneEdge reports the deployment latency of its whole control plane, and Kurma combines its provisioning stages into one quantity,

none attributing the measured time to an individual handoff [11, 37, 40, 42]. Third, the delay is never materialised as an experimental variable: the interfaces of the demand-to-capacity chain are acknowledged and, in places, combined, but no reviewed work varies one interface while holding the others fixed and traces the effect through a stateful service. The closest reviewed precedent is AdapPF, which varies monitoring freshness and measures a downstream consequence, but only for a single consumer, a scheduler [20]. The separation that produces these delays is not an artefact of one platform family but the shared default of the areas reviewed; what varies across them is only how far each acknowledges the boundaries it imposes [45].

Table 2.5 condenses these findings into the three interfaces and the question each leaves open.

TABLE 2.5. The three interfaces of the demand-to-capacity chain and the questions they leave open.

Interface	Open question
Telemetry delivery to decision (RQ1)	Does a controller behave differently when demand evidence arrives late but complete, versus when intermediate observations are absent?
Decision to capacity action (RQ2)	When compute and storage actions are both available, how much does selecting the action matched to an observed compute-bound or data-access-bound bottleneck change targeted-tier relief, service recovery, and resource efficiency relative to fixed single-tier policies, and where do the effects concentrate?
Ready backend to traffic (RQ3)	How does readiness-admission coordination affect the delay from verified readiness to usable capacity and its transition-window service consequence?

The research questions of 1.3 are the experimental form of Table 2.5. This dissertation does not argue that the coordination gap is universal, only that, within the reviewed literature, the interfaces that compose it have not been isolated as independent variables.

The evaluation follows the same per-interface discipline. Each research question varies one interface while holding the other two constant, and the demand-to-capacity chain is treated as a sequence of separately measured interfaces, whose aggregate cost this dissertation deliberately does not claim.

The questions also sit between the surveyed fields rather than inside any of them: each area’s abstraction serves its own question. Chapter 3 presents the platform designed to make them askable and measurable.

[This page is intentionally left blank.]

Architecture and Design of the Proposed Solution for Edge Services

This chapter presents the architecture and design of the proposed cross-layer SDN orchestration platform for stateful edge services. The design provides an experimental environment in which traffic steering, controller-side telemetry consumption, and elasticity coordination operate within each domain’s controller process. The delivery of aggregated telemetry and the lifecycle transitions from capacity action to traffic admission remain explicit observation points. It is not intended to establish the general superiority of a particular orchestration platform; the evaluation of the mechanisms described in this chapter is presented in Chapter 5.

Section 3.1 defines the requirements that guided the proposed solution. Section 3.2 presents the two-domain architecture and its communication paths.

3.1. Design Requirements

The platform was designed to support stateful edge services whose demand and resource requirements vary over time. Its design is guided by the following requirements.

- DR1: Cross-layer coordination.** Within each domain, traffic steering, telemetry consumption, topology awareness, and resource lifecycle management must exchange controller state through the same process. Aggregated telemetry must reach the controller through a structured, configurable delivery interface. Peer controllers must share the topology state needed for cross-domain awareness.
- DR2: Separability of control interfaces.** The delivery of demand evidence, the selection of a capacity action, and the admission of ready capacity to traffic must remain independently configurable.
- DR3: Independent compute and storage elasticity.** The platform must distinguish an increase or decrease in edge-service capacity from an increase or decrease in storage capacity, allowing each resource tier to be managed separately.
- DR4: Lifecycle-safe admission.** Provisioned resources must not be regarded as usable capacity until the relevant readiness condition has been reached and the resource has been admitted to the appropriate routing path.
- DR5: Fast-path isolation.** Packet handling must remain separate from slow infrastructure operations, such as container creation, replica preparation, and resource removal.
- DR6: End-to-end observability.** The platform must record the path from demand observation to the first successful request served by newly available capacity.

3.2. Architecture

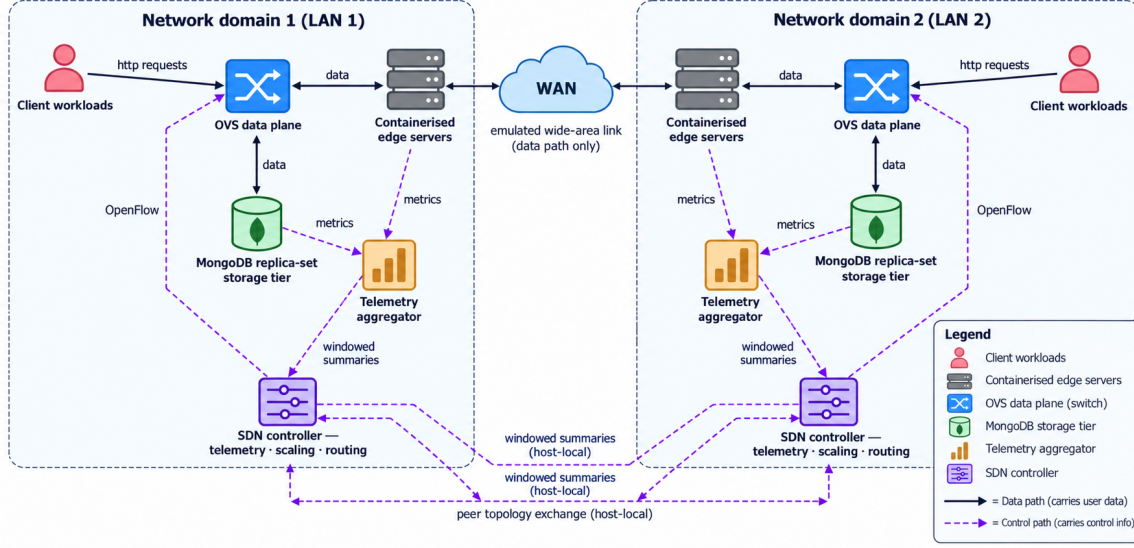


FIGURE 3.1. Architecture of the proposed platform: two network domains whose clients, data planes, edge servers, storage tiers, and aggregators are coordinated by peer SDN controllers, with host-local control paths and an emulated wide-area data link between the domains.

3.2.1. Two-Domain Organisation

The platform is organised as two network domains connected by an emulated wide-area data link. Each domain contains client workloads, an Open vSwitch (OVS) data plane, a pool of containerised edge servers, a MongoDB replica-set storage tier, a telemetry aggregator, and an SDN controller. The peer controllers exchange topology information, and each consumes the telemetry summaries published by both domain aggregators. Each controller runs traffic steering, telemetry consumption, and elasticity coordination in the same process for its own domain.

The emulated wide-area link represents cross-domain data traffic. In contrast, telemetry delivery and controller-to-controller topology exchange use host-local control channels in the experimental environment. This distinction separates data-path delay from control-path communication.

3.2.2. Virtual IP Data Paths

Clients access edge services through a per-domain Virtual IP (VIP). The virtual service addresses are `VIP_SERVER_N1` and `VIP_SERVER_N2`, collectively referred to as `VIP_SERVER`. Edge servers use the virtual data addresses `VIP_DATA_N1` and `VIP_DATA_N2`, collectively referred to as `VIP_DATA`, to access the storage tier. This separation represents service routing and data access as distinct routing surfaces.

When a flow addressed to a virtual IP is first observed, the controller selects an eligible backend and configures the OVS data plane to forward the flow. Subsequent matching packets are handled in the data plane without requiring repeated controller

intervention. Service backend selection can include eligible backends in either domain when the topology permits. The architecture also supports cross-domain data access; however, the final evaluation workload keeps content ownership local so that this path remains a platform capability rather than an experimental treatment.

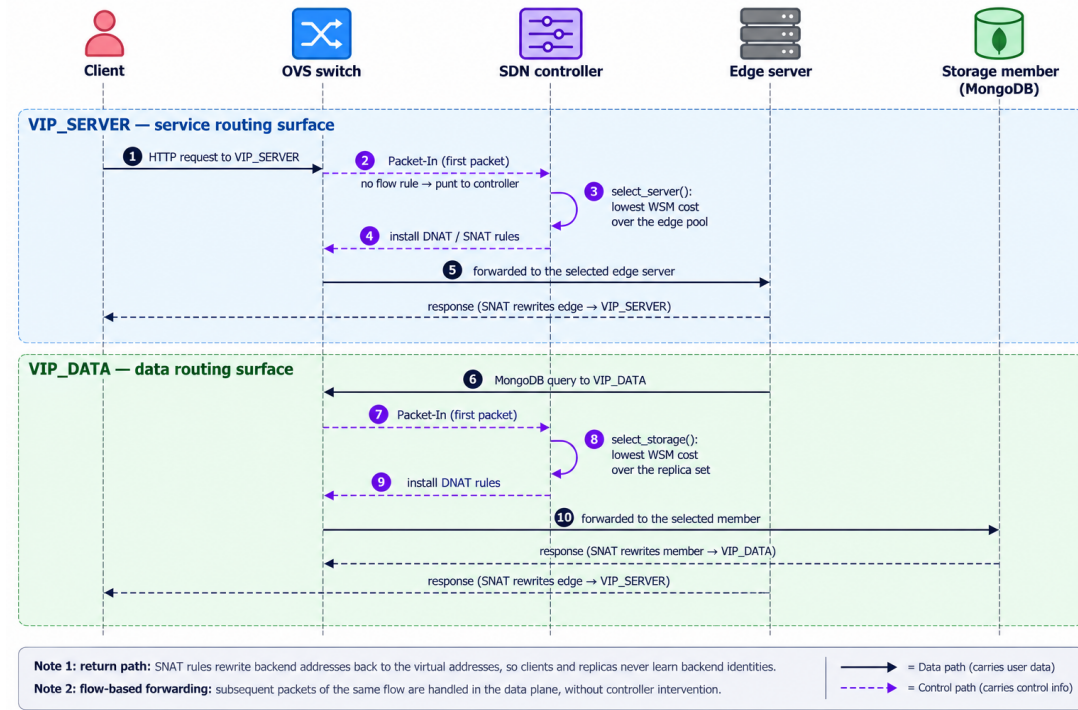


FIGURE 3.2. Double-VIP routing sequence: a client request addressed to VIP_SERVER triggers backend selection in the controller, and the edge server’s storage access through VIP_DATA triggers the selection of a replica-set member.

3.2.3. Control and Observation Paths

Edge servers and storage components emit resource and request observations to their local telemetry aggregator through ZMQ. Each aggregator forms windowed summaries that are consumed by the controller. These summaries inform both backend selection and elasticity decisions, while topology exchange maintains awareness of peer-domain resources. The aggregation-to-controller link is the configurable telemetry-delivery interface.

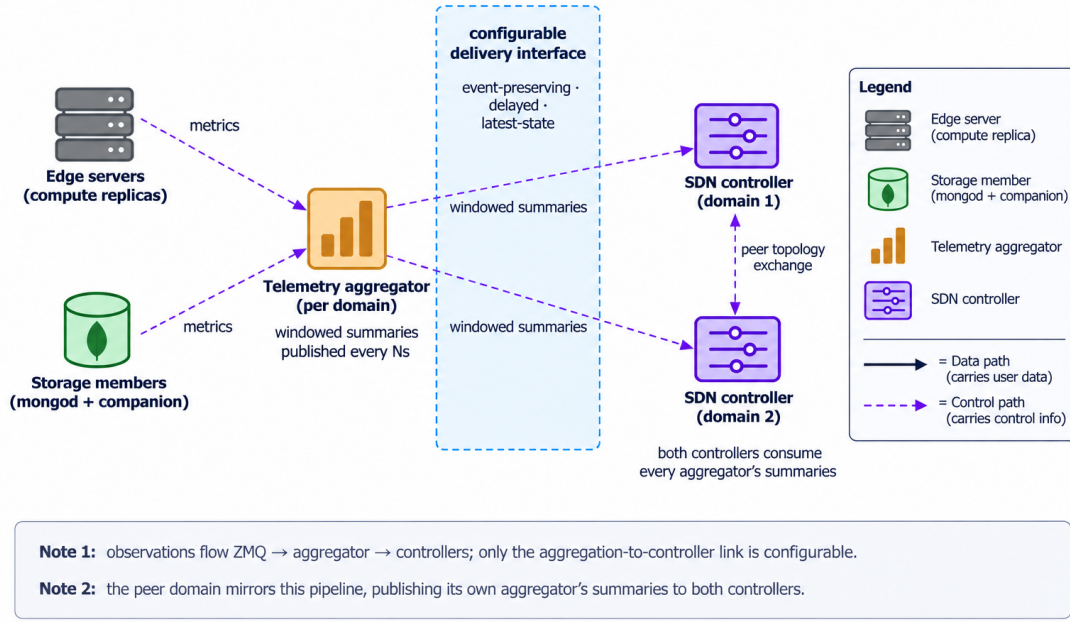


FIGURE 3.3. Control and observation paths: edge servers and storage members publish observations to the per-domain telemetry aggregator, whose windowed summaries reach both peer controllers through a configurable delivery interface.

The architecture therefore separates the request data path, the telemetry observation path, and the slow infrastructure-management path while retaining their coordination within each domain's controller process. The concrete implementation of these paths is described in Chapter 4.

The deployment evaluated in this dissertation runs the two domains on a single host, with an emulated wide-area link; no performance claim is made for a real geo-distributed wide-area deployment. The testbed is described in the experimental setup of Chapter 5.

3.3. Distributed Elastic Allocation of Resources

The platform scales capacity along two axes: edge-service replicas, which serve requests, and storage members, which hold the service state. Each axis has its own observation signal, its own action, and its own admission and release procedure, so that pressure on the service tier and pressure on the data tier can be addressed without conflating the two. The two axes are described in turn.

3.3.1. Scaling Policy

The policy is instantiated once per resource tier, with tier-specific signals and parameters. Each tier is observed through two signals: a utilisation signal and a latency signal. For each telemetry window w , let $c(w)$ denote the window's utilisation and $t(w)$ its typical latency. Each signal is first mapped onto the unit interval: a value at or below its floor contributes nothing, a value at or above the floor plus its span contributes the maximum,

and intermediate values contribute proportionally, so that a single extreme window cannot dominate the score. The two normalised components are combined with weights w_{util} and w_{lat} into the degradation score

$$\begin{aligned} s(w) &= \frac{w_{\text{util}} \hat{c}(w) + w_{\text{lat}} \hat{t}(w)}{w_{\text{util}} + w_{\text{lat}}}, \\ \hat{c}(w) &= \min \left(1, \max \left(0, \frac{c(w) - c_0}{c_\Delta} \right) \right), \\ \hat{t}(w) &= \min \left(1, \max \left(0, \frac{t(w) - t_0}{t_\Delta} \right) \right), \end{aligned} \tag{3.1}$$

which therefore always lies between zero and one, where:

- $c(w)$ – utilisation of the scored tier observed in window w ;
- $t(w)$ – typical latency of the scored tier observed in window w ;
- c_0, t_0 – signal floors: at or below these values a signal contributes nothing;
- c_Δ, t_Δ – signal spans: at or above floor plus span a signal contributes the maximum;
- $\hat{c}(w), \hat{t}(w)$ – normalised components of window w , each in $[0, 1]$;
- $w_{\text{util}}, w_{\text{lat}}$ – configurable weights of the utilisation and latency components.

The floors, spans, and weights are configuration parameters, set independently for each tier: for the service tier the signals are CPU utilisation and request-processing latency, while for the data tier they are storage utilisation and data-access latency.

A window is considered degraded when its score reaches an adaptive threshold, and a scale-up is requested only when degradation is sustained across a required number of recent windows. Let $\mathcal{W}_n$ be the set of the last W windows and R the required number of degraded windows; the scale-up condition is

$$\begin{aligned} |\{w \in \mathcal{W}_n : s(w) \geq \theta(k)\}| &\geq R, \\ \theta(k) &= \min(\theta_{\text{base}} + k \Delta\theta + \beta_{\text{peer}}, \theta_{\text{max}}), \end{aligned} \tag{3.2}$$

where:

- $\mathcal{W}_n$ – sliding set of the most recent W windows;
- R – minimum number of degraded windows required to request a scale-up;
- k – number of dynamic units currently in the tier’s pool;
- $\theta(k)$ – adaptive degradation threshold with k dynamic units in the pool;
- θ_{base} – base threshold;
- $\Delta\theta$ – threshold increment per additional dynamic unit;
- θ_{max} – upper bound of the adaptive threshold;
- β_{peer} – addend applied while the peer domain is healthy enough to absorb spillover traffic.

The threshold $\theta(k)$ adapts upward as capacity grows, so each additional unit faces a higher bar, bounded by θ_{max} , and β_{peer} raises it slightly while the peer domain is healthy enough to absorb spillover traffic. A cooldown after each action and a per-tier cap on the number of units keep the number of scale actions bounded.

A window is declared idle when both signals fall below their idle thresholds,

$$c(w) < \tau_c \quad \text{and} \quad t(w) < \tau_t, \quad (3.3)$$

where:

- τ_c, τ_t – idle thresholds of the utilisation and latency signals;
- $R_\downarrow, W_\downarrow$ – required idle windows and window size of the scale-down rule.

A scale-down is requested only when idleness is sustained across $R_\downarrow$ of the last $W_\downarrow$ windows, a sliding window longer than the scale-up one, which prevents a transient data-bound stall from being misread as idle capacity. The concrete parameter values used in the evaluation are reported in the experimental setup of Chapter 5.

3.3.2. Services

Service capacity is a per-domain pool of containerised edge servers, each capable of serving any request addressed to the domain’s virtual service address. All storage access performed by a service replica goes through the domain’s virtual data address, so a replica carries no physical knowledge of the storage tier. Service scale-up applies the policy defined in 3.3.1, with CPU utilisation and request-processing latency as signals.

A newly provisioned service replica is recorded through an explicit registration step; the replica becomes eligible for backend selection only once it is visible in the routing pool used by the selector. The point at which registration occurs relative to application readiness is itself a configurable design dimension: the replica may be admitted immediately upon creation, or admission may be deferred until a readiness condition is verified, which is the mechanism examined for the admission interface of the demand-to-capacity chain.

Service scale-down removes the least disruptive eligible replica, applying the idle predicate defined in 3.3.1. Removal proceeds in two phases: the replica is first withdrawn from the routing pool and signalled to finish its in-flight requests, and only after it drains is its network attachment removed. A pending drain can be cancelled if load rebounds before the replica exits, favouring fast recovery over aggressive reclamation.

3.3.3. Data

The replica set is the unit in which storage capacity is scaled. The data tier of each domain runs as a MongoDB replica set whose primary serves the clients’ writes and whose secondaries replicate the data and absorb the remainder of the traffic. Storage scale-up is a membership change: the controller provisions a new storage member, joins it to the set, and admits it to the data path only once the member has reached the state at which it can serve reads, so that traffic is never directed to a member that cannot yet serve it. The preparation may also be decoupled from the reaction: a ready member can be held in reserve outside the routing pool and activated when a data action fires, removing the initial readiness wait from the reaction path, with a new reserve prepared immediately afterwards.

Storage scale-down removes an idle member while the remaining members keep serving. Only members added by the platform are eligible for removal, so the set’s original membership is never disturbed, and the decision applies the idle predicate defined in 3.3.1 with the data tier’s utilisation and latency signals, evaluated over sustained low data-access latency and utilisation.

Beyond the membership change, the platform defines three data-placement tiers, named here for completeness: Tier 0, direct remote reads over the virtual data path; Tier 1, a partial hot copy held in the consumer domain; and Tier 2, the replica-set extension described above. These locality mechanisms are platform capabilities rather than evaluated treatments: they are disabled or held constant wherever an evaluation requires their isolation, and the capacity action evaluated in this dissertation is the same-domain membership change.

3.4. Monitoring and Decision Engine

Observation begins at the producers. Each service replica emits a per-request record of its resource state and request experience, and each storage member’s companion process emits periodic replication and resource state. A per-domain aggregator collects these records and publishes a windowed summary at a fixed cadence, so that time is observed as a regular sequence of windows and every window, including idle ones, carries an evaluation outcome.

The delivery of these summaries to the controller is an explicit, configurable interface. The default delivery pushes each completed window as it is published. Alternative delivery modes supply the same windowed evidence in other forms, such as periodic retrieval of only the most recent window or ordered replay of a retained window history, so that the effect of delayed or incomplete demand evidence can be studied while the aggregation, the workload, and the decisions remain identical.

Inside the controller, the summaries feed two consumers. The first is the routing state: per-backend metrics update the state that backend selection uses. The second is the decision engine: per-domain degradation scores, computed per resource tier from bounded utilisation and typical request latency, are evaluated against adaptive thresholds over sliding windows. When a tier’s predicate is sustained, the decision engine emits a typed, prioritised request for a capacity action, with data-side actions given precedence over service-side actions. Scale-down requests arise from separate idle predicates.

The decision engine may also be configured to commit to a single action per window when several tiers signal simultaneously, which is the selection interface of the demand-to-capacity chain. Lifecycle signals, such as the completion of a drain or the readiness of a storage member, reach the same decision path as control events, so that admission and release follow the same disciplined flow as the actions themselves.

3.5. Control Workflow

The control workflow separates three execution contexts that share one process but never share blocking: the packet context, which handles flow decisions and never waits on slow work; the observation context, which evaluates telemetry windows and emits action requests; and the action context, which performs infrastructure mutations such as provisioning, drain, and removal.

The workflow traverses the demand-to-capacity chain introduced in 1.1. A change in demand alters the request experience reported by the service replicas; the window that contains that change is published and delivered; the decision engine evaluates it and, if the tier predicate is sustained, emits an action request; the action context executes it; the provisioned unit reaches its readiness state and is admitted to the corresponding routing pool; and from the next packet the data plane can direct traffic to it, which completes the chain when the new unit serves its first successful request. Each step of this chain is instrumented, so the timeline from demand observation to usable capacity is recorded end to end.

State crosses these contexts through shared structures rather than through direct calls. The observation context writes the telemetry state and the action requests; the routing pools are updated as units are provisioned, readied, and removed; and the packet context reads both on each flow decision. Because the packet context is never blocked by the action context, a long-running mutation cannot stall request steering, and because the action requests are queued and prioritised, detection is never blocked by execution.

Across domains, the peer controllers exchange topology state so that each controller is aware of the peer domain's resources, while acting on its own domain's demand. The concrete realisation of this design, the technologies that implement each context, and the validation of the resulting platform are the subject of Chapter 4; the experimental characterisation of the three interfaces is presented in Chapter 5.

CHAPTER 4

Implementation of Proposed Solution

4.1. Experimental infrastructure

4.2. Distributed Elastic Allocation of Resources

4.2.1. Services

4.2.2. Data

4.3. Monitoring and Decision Engine

4.4. Control Workflow

4.5. Implementation Validation

[This page is intentionally left blank.]

CHAPTER 5

Experimental Results

- 5.1. Experimental Setup
- 5.2. Telemetry Delivery Semantics (RQ1)
- 5.3. Bottleneck-Aware Scaling Action (RQ2)
- 5.4. Readiness Admission to Traffic (RQ3)
- 5.5. Network Performance
- 5.6. Scalability Analysis
- 5.7. Discussion: Synthesis Across Interfaces

[This page is intentionally left blank.]

CHAPTER 6

Conclusions and Future Work

6.1. Conclusions

6.2. Research Contributions

6.3. Limitations

6.4. Future Research Directions

[This page is intentionally left blank.]

References

- [1] International Telecommunication Union, “Facts and figures 2025: Internet traffic,” ITU, Oct. 2025. Accessed: Jul. 13, 2026. [Online]. Available: <https://www.itu.int/itu-d/reports/statistics/2025/10/15/ff25-internet-traffic/>
- [2] D. Gurung, S. M. Z. U. Rashid, Z. u. Abdeen, and S. Rath, “Cloud revolution: Tracing the origins and rise of cloud computing,” in *2026 IEEE 16th Annual Computing and Communication Workshop and Conference (CCWC)*, Las Vegas, NV, USA, Jan. 2026, pp. 1100–1106. DOI: 10.1109/CCWC67433.2026.11393790 [Online]. Available: <https://doi.org/10.1109/CCWC67433.2026.11393790>
- [3] T. Gaudiaut, *Mobile accounts for nearly 60 percent of web traffic*, Statista Chart of the Day, Data from StatCounter. Licensed under CC BY-ND 3.0, Mar. 2026. Accessed: Jul. 19, 2026. [Online]. Available: <https://www.statista.com/chart/35951/share-of-mobile-device-website-traffic-worldwide-since-2010/>
- [4] K. Cao, Y. Liu, G. Meng, and Q. Sun, “An overview on edge computing research,” *IEEE Access*, vol. 8, pp. 85 714–85 728, 2020. DOI: 10.1109/ACCESS.2020.2991734 [Online]. Available: <https://doi.org/10.1109/ACCESS.2020.2991734>
- [5] M. Satyanarayanan, “The emergence of edge computing,” *Computer*, vol. 50, no. 1, pp. 30–39, Jan. 2017. DOI: 10.1109/MC.2017.9 [Online]. Available: <https://doi.org/10.1109/MC.2017.9>
- [6] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, “Resource scheduling in edge computing: A survey,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2131–2165, 2021. DOI: 10.1109/COMST.2021.3106401 [Online]. Available: <https://doi.org/10.1109/COMST.2021.3106401>
- [7] M. Breitbach, D. Schäfer, J. Edinger, and C. Becker, “Context-aware data and task placement in edge computing environments,” in *2019 IEEE International Conference on Pervasive Computing and Communications (PerCom)*, Kyoto, Japan, Mar. 2019, pp. 1–10. DOI: 10.1109/PERCOM.2019.8767386 [Online]. Available: <https://doi.org/10.1109/PERCOM.2019.8767386>
- [8] A.-C. Nicolaescu, S. Mastorakis, and I. Psaras, “Store edge networked data (send): A data and performance driven edge storage framework,” in *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*, Vancouver, BC, Canada, May 2021, pp. 1–10. DOI: 10.1109/INFOCOM42981.2021.9488804 [Online]. Available: <https://doi.org/10.1109/INFOCOM42981.2021.9488804>
- [9] I. Pelle, M. Szalay, J. Czentye, B. Sonkoly, and L. Toka, “Cost and latency optimized edge computing platform,” *Electronics*, vol. 11, no. 4, p. 561, Feb. 2022. DOI: 10.

- 3390/electronics11040561 [Online]. Available: <https://doi.org/10.3390/electronics11040561>
- [10] C. Qu, R. N. Calheiros, and R. Buyya, “Auto-scaling web applications in clouds: A taxonomy and survey,” *ACM Computing Surveys*, vol. 51, no. 4, pp. 1–33, Jul. 2018. DOI: 10.1145/3148149 [Online]. Available: <https://doi.org/10.1145/3148149>
 - [11] A. Llorens-Carrodegua, I. Leyva-Pupo, C. Cervelló-Pastor, L. Piñeiro, and S. Siddiqui, “An sdn-based solution for horizontal auto-scaling and load balancing of transparent vnf clusters,” *Sensors*, vol. 21, no. 24, p. 8283, Dec. 2021. DOI: 10.3390/s21248283 [Online]. Available: <https://doi.org/10.3390/s21248283>
 - [12] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–106, Mar. 2004. DOI: 10.2307/25148625 [Online]. Available: <https://doi.org/10.2307/25148625>
 - [13] J. vom Brocke, A. Hevner, and A. Maedche, “Introduction to design science research,” in *Progress in IS*, 2020, pp. 1–13, ISBN: 9783030467807. DOI: 10.1007/978-3-030-46781-4_1 [Online]. Available: https://doi.org/10.1007/978-3-030-46781-4_1
 - [14] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, Dec. 2007. DOI: 10.2753/MIS0742-1222240302 [Online]. Available: <https://doi.org/10.2753/MIS0742-1222240302>
 - [15] F. Liu, G. Tang, Y. Li, Z. Cai, X. Zhang, and T. Zhou, “A survey on edge computing systems and tools,” *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1537–1562, 2019. DOI: 10.1109/JPROC.2019.2920341 [Online]. Available: <https://doi.org/10.1109/JPROC.2019.2920341>
 - [16] A. Nain, S. Sheikh, M. Shahid, and R. Malik, “Resource optimization in edge and sdn-based edge computing: A comprehensive study,” *Cluster Computing*, vol. 27, no. 5, pp. 5517–5545, Feb. 2024. DOI: 10.1007/s10586-023-04256-8 [Online]. Available: <https://doi.org/10.1007/s10586-023-04256-8>
 - [17] A. C. Baktir, A. Ozgovde, and C. Ersoy, “How can edge computing benefit from software-defined networking: A survey, use cases, and future directions,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2359–2391, 2017. DOI: 10.1109/COMST.2017.2717482 [Online]. Available: <https://doi.org/10.1109/COMST.2017.2717482>
 - [18] Q. Waseem, W. Isnii Sofiah Wan Din, A. Aminuddin, M. Hussain Mohammed, and R. F. Alfa Aziza, “Software-defined networking (sdn): A review,” in *2022 5th International Conference on Information and Communications Technology (ICOIACT)*, Yogyakarta, Indonesia, Aug. 2022, pp. 30–35. DOI: 10.1109/ICOIACT55506.2022.9972067 [Online]. Available: <https://doi.org/10.1109/ICOIACT55506.2022.9972067>

- [19] N. Yaseen, "From counters to telemetry: A survey of programmable network-wide monitoring," *Network*, vol. 5, no. 3, p. 38, Sep. 2025. DOI: 10.3390/network5030038 [Online]. Available: <https://doi.org/10.3390/network5030038>
- [20] C.-K. Huang and G. Pierre, "Adappf: Self-adaptive scrape interval for monitoring in geo-distributed cluster federations," in *2023 IEEE Symposium on Computers and Communications (ISCC)*, Gammarth, Tunisia, Jul. 2023, pp. 417–423. DOI: 10.1109/ISCC58397.2023.10218080 [Online]. Available: <https://doi.org/10.1109/ISCC58397.2023.10218080>
- [21] C.-K. Huang and G. Pierre, "Aggregate monitoring for geo-distributed kubernetes cluster federations," *IEEE Transactions on Cloud Computing*, vol. 12, no. 4, pp. 1449–1462, Oct. 2024. DOI: 10.1109/TCC.2024.3482574 [Online]. Available: <https://doi.org/10.1109/TCC.2024.3482574>
- [22] H. Zhou and C. H. Yong, "Implement hpa for nginx service using custom metrics under kubernetes framework," *IEEE Access*, vol. 12, pp. 189 722–189 734, 2024. DOI: 10.1109/ACCESS.2024.3509876 [Online]. Available: <https://doi.org/10.1109/ACCESS.2024.3509876>
- [23] M.-H. Hung, C.-C. Teng, C.-P. Chuang, C.-S. Hsu, J.-W. Gong, and M.-C. Chen, "A sdn controller monitoring architecture for 5g backhaul networks," in *2022 23rd Asia-Pacific Network Operations and Management Symposium (APNOMS)*, Takamatsu, Japan, Sep. 2022, pp. 1–4. DOI: 10.23919/APNOMS56106.2022.9919988 [Online]. Available: <https://doi.org/10.23919/APNOMS56106.2022.9919988>
- [24] M. Usman, S. Ferlin, A. Brunstrom, and J. Taheri, "A survey on observability of distributed edge & container-based microservices," *IEEE Access*, vol. 10, pp. 86 904–86 919, 2022. DOI: 10.1109/ACCESS.2022.3193102 [Online]. Available: <https://doi.org/10.1109/ACCESS.2022.3193102>
- [25] M. I. S. El Mrabet, A. Malki, and M. Malki, "A comprehensive survey and taxonomy of autoscaling techniques in cloud and edge environments," in *2025 Fourth International Conference on Theoretical and Applicative Aspects of Computer Science (ICTAACS)*, Skikda, Algeria, Dec. 2025, pp. 1–8. DOI: 10.1109/ICTAACS69003.2025.11399335 [Online]. Available: <https://doi.org/10.1109/ICTAACS69003.2025.11399335>
- [26] S. Taherizadeh and V. Stankovski, "Auto-scaling applications in edge computing," in *Proceedings of the International Conference on Big Data and Internet of Thing*, London United Kingdom, Dec. 2017, pp. 158–163. DOI: 10.1145/3175684.3175709 [Online]. Available: <https://doi.org/10.1145/3175684.3175709>
- [27] L. Toka, G. Dobreff, B. Fodor, and B. Sonkoly, "Machine learning-based scaling management for kubernetes edge clusters," *IEEE Transactions on Network and Service Management*, vol. 18, no. 1, pp. 958–972, Mar. 2021. DOI: 10.1109/TNSM.2021.3052837 [Online]. Available: <https://doi.org/10.1109/TNSM.2021.3052837>

- [28] L. Ju, P. Singh, and S. Toor, "Proactive autoscaling for edge computing systems with kubernetes," in *Proceedings of the 14th IEEE/ACM International Conference on Utility and Cloud Computing Companion*, Leicester United Kingdom, Dec. 2021, pp. 1–8. DOI: 10.1145/3492323.3495588 [Online]. Available: <https://doi.org/10.1145/3492323.3495588>
- [29] K. Cheng, S. Zhang, C. Tu, X. Shi, Z. Yin, S. Lu, Y. Liang, and Q. Gu, "Proscale: Proactive autoscaling for microservice with time-varying workload at the edge," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 4, pp. 1294–1312, Apr. 2023. DOI: 10.1109/TPDS.2023.3238429 [Online]. Available: <https://doi.org/10.1109/TPDS.2023.3238429>
- [30] Y. Li, H. Zhang, W. Tian, and H. Ma, "Joint optimization of auto-scaling and adaptive service placement in edge computing," in *2021 IEEE 27th International Conference on Parallel and Distributed Systems (ICPADS)*, Beijing, China, Dec. 2021, pp. 923–930. DOI: 10.1109/ICPADS53394.2021.00121 [Online]. Available: <https://doi.org/10.1109/ICPADS53394.2021.00121>
- [31] A. Y. Nikraves, S. A. Ajila, and C.-H. Lung, "The impact of database layer on auto-scaling decisions in a 3-tier web services cloud resource provisioning," in *2017 IEEE 41st Annual Computer Software and Applications Conference (COMPSAC)*, Turin, Jul. 2017, pp. 401–406. DOI: 10.1109/COMPSAC.2017.235 [Online]. Available: <https://doi.org/10.1109/COMPSAC.2017.235>
- [32] J. Dogani, R. Namvar, and F. Khunjush, "Auto-scaling techniques in container-based cloud and edge/fog computing: Taxonomy and survey," *Computer Communications*, vol. 209, pp. 120–150, Sep. 2023. DOI: 10.1016/j.comcom.2023.06.010 [Online]. Available: <https://doi.org/10.1016/j.comcom.2023.06.010>
- [33] S. Xie, J. Wang, B. Li, Z. Zhang, D. Li, and P. C. K. Hung, "Pbscaler: A bottleneck-aware autoscaling framework for microservice-based applications," *IEEE Transactions on Services Computing*, vol. 17, no. 2, pp. 604–616, Mar. 2024. DOI: 10.1109/TSC.2024.3376202 [Online]. Available: <https://doi.org/10.1109/TSC.2024.3376202>
- [34] C. Li, J. Bai, Y. Ge, and Y. Luo, "Heterogeneity-aware elastic provisioning in cloud-assisted edge computing systems," *Future Generation Computer Systems*, vol. 112, pp. 1106–1121, Nov. 2020. DOI: 10.1016/j.future.2020.06.022 [Online]. Available: <https://doi.org/10.1016/j.future.2020.06.022>
- [35] A. A. Neghabi, N. Jafari Navimipour, M. Hosseinzadeh, and A. Rezaee, "Load balancing mechanisms in the software defined networks: A systematic and comprehensive review of the literature," *IEEE Access*, vol. 6, pp. 14 159–14 178, 2018. DOI: 10.1109/ACCESS.2018.2805842 [Online]. Available: <https://doi.org/10.1109/ACCESS.2018.2805842>
- [36] M. R. Belgaum, S. Musa, M. M. Alam, and M. M. Su'ud, "A systematic review of load balancing techniques in software-defined networking," *IEEE Access*, vol. 8,

- pp. 98 612–98 636, 2020. DOI: 10.1109/ACCESS.2020.2995849 [Online]. Available: <https://doi.org/10.1109/ACCESS.2020.2995849>
- [37] K. L. Bogdanov, W. Reda, G. Q. Maguire Jr., D. Kostić, and M. Canini, “Fast and accurate load balancing for geo-distributed storage systems,” in *Proceedings of the ACM Symposium on Cloud Computing*, Carlsbad CA USA, Oct. 2018, pp. 386–400. DOI: 10.1145/3267809.3267820 [Online]. Available: <https://doi.org/10.1145/3267809.3267820>
- [38] A. Wang, Z. Zha, Y. Guo, and S. Chen, “Software-defined networking enhanced edge computing: A network-centric survey,” *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1500–1519, Aug. 2019. DOI: 10.1109/JPROC.2019.2924377 [Online]. Available: <https://doi.org/10.1109/JPROC.2019.2924377>
- [39] R. C. Sofia, D. Dykeman, P. Urbanetz, A. Galal, and D. A. Dave, “Dynamic, context-aware cross-layer orchestration of containerized applications,” *IEEE Access*, vol. 11, pp. 93 129–93 150, 2023. DOI: 10.1109/ACCESS.2023.3307026 [Online]. Available: <https://doi.org/10.1109/ACCESS.2023.3307026>
- [40] J. Okwuibe, J. Haavisto, E. Harjula, I. Ahmad, and M. Ylianttila, “Sdn enhanced resource orchestration of containerized edge applications for industrial iot,” *IEEE Access*, vol. 8, pp. 229 117–229 131, 2020. DOI: 10.1109/ACCESS.2020.3045563 [Online]. Available: <https://doi.org/10.1109/ACCESS.2020.3045563>
- [41] H. Peng, Q. Ye, and X. S. Shen, “Sdn-based resource management for autonomous vehicular networks: A multi-access edge computing approach,” *IEEE Wireless Communications*, vol. 26, no. 4, pp. 156–162, Aug. 2019. DOI: 10.1109/MWC.2019.1800371 [Online]. Available: <https://doi.org/10.1109/MWC.2019.1800371>
- [42] E. Saurez, H. Gupta, A. Daglis, and U. Ramachandran, “Oneedge: An efficient control plane for geo-distributed infrastructures,” in *Proceedings of the ACM Symposium on Cloud Computing*, Seattle WA USA, Nov. 2021, pp. 182–196. DOI: 10.1145/3472883.3487008 [Online]. Available: <https://doi.org/10.1145/3472883.3487008>
- [43] M. A. Tamiru, G. Pierre, J. Tordsson, and E. Elmroth, “Mck8s: An orchestration platform for geo-distributed multi-cluster environments,” in *2021 International Conference on Computer Communications and Networks (ICCCN)*, Athens, Greece, Jul. 2021, pp. 1–10. DOI: 10.1109/ICCCN52240.2021.9522318 [Online]. Available: <https://doi.org/10.1109/ICCCN52240.2021.9522318>
- [44] I. Čilić, P. Krivić, I. Podnar Žarko, and M. Kušek, “Performance evaluation of container orchestration tools in edge computing environments,” *Sensors*, vol. 23, no. 8, p. 4008, Apr. 2023. DOI: 10.3390/s23084008 [Online]. Available: <https://doi.org/10.3390/s23084008>
- [45] H. Tabatabaee Malazi, S. R. Chaudhry, A. Kazmi, A. Palade, C. Cabrera, G. White, and S. Clarke, “Dynamic service placement in multi-access edge computing: A systematic literature review,” *IEEE Access*, vol. 10, pp. 32 639–32 688, 2022. DOI: 10.

1109/ACCESS.2022.3160738 [Online]. Available: <https://doi.org/10.1109/ACCESS.2022.3160738>